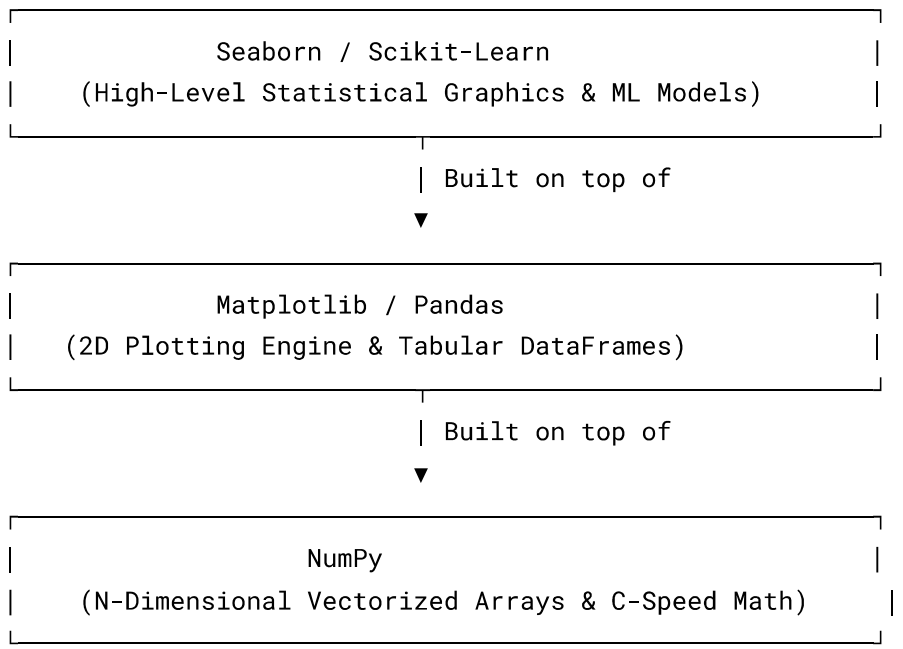
End-Semester Examination Study Guide: Data Science & Visualization Libraries (Matplotlib, Seaborn, NumPy, Pandas) and Package Management

In the Python ecosystem, external libraries and modules extend the language's core functionality to turn raw Python into a powerful platform for data analysis, scientific computing, and visualization.

Part 1: Theoretical Foundations & Library Architecture

1. The Data Science Library Hierarchy

Python's data science stack relies on specialized, layered libraries:



- **NumPy (Numerical Python)**: Provides fast (N)-dimensional array objects (`ndarray`) and vectorized mathematical operations, avoiding sluggish Python `for` loops.
- **Pandas**: Built on top of NumPy, Pandas provides structured tabular data containers called **Series** (1D) and **DataFrames** (2D) for cleaning, slicing, filtering, and aggregating structured datasets.
- **Matplotlib (`matplotlib.pyplot`)**: The core, low-level 2D plotting library in Python. It provides fine-grained control over figures, axes, line styles, titles, legends, and grid layouts.
- **Seaborn**: A high-level visualization library built directly on top of Matplotlib. It integrates seamlessly with Pandas DataFrames and provides pre-styled statistical plots (e.g., heatmaps, violin plots, regression trends) with attractive visual defaults.

2. Package Management (`pip`) and Dependency Isolation (`venv`)

External libraries are not part of Python's standard library ("batteries included") and must be downloaded from the **Python Package Index (PyPI)**.

- **`pip` Utility**: The package installer for Python.
  - `pip install pandas matplotlib seaborn` : Installs packages globally or locally.
  - `pip list` : Lists all currently installed packages and their exact versions.
  - `pip freeze > requirements.txt` : Exports current environment dependencies to a plain text file.
  - `pip install -r requirements.txt` : Recreates the exact environment on another machine.
- **Virtual Environments (`venv`)**: Isolated Python environments that prevent version conflicts between different projects.
  - Creation: `python -m venv myenv`
  - Activation (PowerShell/Bash): `env\Scripts\activate.ps1` or `source env/bin/activate`
  - Deactivation: `deactivate`

Part 2: 100-Mark End-Semester Exam Practice Suite (5 × 20M)

Question 1 (20 Marks): External Modules, Virtual Environments (`venv`), and Import Safety

(a) [5 Marks] Core Concepts

1. Differentiate between `import module` and `import module as alias` .
2. Explain why virtual environments (`venv`) are mandatory when deploying Python data applications.

(b) [5 Marks] Package Management Commands

Write the exact CLI terminal commands to:

1. Create a virtual environment named `exam_env` .
2. Install `pandas` and `matplotlib` inside the environment.
3. Generate a `requirements.txt` dependency log.

© [5 Marks] Simple Baseline Solution: Package Loader Script

```
# Baseline Script: Standard Module Import
import sys

def check_environment():
    print("Python Executable Path:", sys.executable)
    print("Python Version:", sys.version)

if __name__ == "__main__":
    check_environment()
```

(d) [5 Marks] Complex Improvement: Modular OOP Environment Inspector with Explicit Exception Guardrails

```
import sys
import importlib

# --- Custom Exception Class ---
class MissingDependencyError(Exception):
    """Raised when a required data science library is missing from the active enviror
    def __init__(self, package_name):
        self.package_name = package_name
        self.message = f"CRITICAL: Package '{package_name}' is missing! Install via '
        super().__init__(self.message)

# --- Modular OOP Inspector Class ---
class EnvironmentValidator:
    def __init__(self, required_packages):
        # Type Check
        if not isinstance(required_packages, (list, tuple, set)):
            raise TypeError("Required packages must be provided as a list, tuple, or
        self.required_packages = required_packages
        self.status_report = {}

    def validate_all(self):
        """Scans environment and verifies availability of required modules."""
        for pkg in self.required_packages:
            try:
                # Dynamic import check
                mod = importlib.import_module(pkg)
                version = getattr(mod, '__version__', 'Unknown Version')
                self.status_report[pkg] = ("INSTALLED", version)
            except ImportError:
                self.status_report[pkg] = ("MISSING", "N/A")
                raise MissingDependencyError(pkg)
```

```
    def print_summary(self):
        print("=" * 60)
        print(f"{'PACKAGE':<20} | {'STATUS':<15} | {'VERSION':<15}")
        print("=" * 60)
        for pkg, (status, ver) in self.status_report.items():
            print(f"{pkg:<20} | {status:<15} | {ver:<15}")
        print("=" * 60)

if __name__ == "__main__":
    dependencies = ["numpy", "pandas", "matplotlib", "seaborn"]
    validator = EnvironmentValidator(dependencies)

    try:
        validator.validate_all()
    except MissingDependencyError as ex:
        print(f"\n[Environment Failure] {ex}")
    except TypeError as ex:
        print(f"\n[Configuration Error] {ex}")
    else:
        print("\n[Environment Success] All required data libraries are available.")
    finally:
        validator.print_summary()
```

Question 2 (20 Marks): Data Structures & Statistical Processing with Input Validation

(a) [5 Marks] List vs. Vectorized Array Operations

Explain the mathematical and execution differences between multiplying a Python list by an integer (`* 2`) versus performing element-wise scalar multiplication on numerical datasets.

(b) [5 Marks] Exception Hazards in Dataset Ingestion

Detail the causes of `ValueError` , `TypeError` , and `ZeroDivisionError` during statistical aggregation operations.

© [5 Marks] Simple Baseline Solution: Statistical Summarizer

```
def calculate_stats(data_list):
    total = sum(data_list)
    count = len(data_list)
    mean = total / count
    return mean
```

```
data = [10.5, 20.0, 30.5, 40.0]
print("Average:", calculate_stats(data))
```

(d) [5 Marks] Complex Improvement: OOP Data Analyzer Class with Full Exception Guards

```
import math

class EmptyDatasetError(Exception):
    """Custom exception raised when processing an empty data container."""
    def __init__(self):
        super().__init__("Cannot compute statistical metrics on an empty dataset.")

class DataAnalyzer:
    def __init__(self, raw_data):
        self._data = []
        self.set_data(raw_data)

    def set_data(self, raw_data):
        """Validates and loads raw numerical sequence."""
        if not isinstance(raw_data, (list, tuple)):
            raise TypeError("Dataset input must be a list or tuple.")

        cleaned = []
        for index, item in enumerate(raw_data):
            try:
                val = float(item)
                if math.isnan(val) or math.isinf(val):
                    raise ValueError(f"Invalid numerical value at index {index}: {ite
                cleaned.append(val)
            except (ValueError, TypeError) as ex:
                raise ValueError(f"Data conversion failed at index {index} ('{item}')

        if not cleaned:
            raise EmptyDatasetError()

        self._data = cleaned

    def compute_mean(self):
        if not self._data:
            raise EmptyDatasetError()
        return sum(self._data) / len(self._data)

    def compute_std_dev(self):
        mean = self.compute_mean()
```

```
        variance = sum((x - mean) ** 2 for x in self._data) / len(self._data)
        return math.sqrt(variance)

    def get_summary(self):
        return {
            "count": len(self._data),
            "min": min(self._data),
            "max": max(self._data),
            "mean": round(self.compute_mean(), 2),
            "std_dev": round(self.compute_std_dev(), 2)
        }

if __name__ == "__main__":
    test_samples = [
        [12.5, 15.0, 18.2, 22.1, 30.0],  # Valid dataset
        [],                              # Triggers EmptyDatasetError
        [10.0, "invalid_num", 30.0]      # Triggers ValueError
    ]

    for idx, sample in enumerate(test_samples, start=1):
        print(f"\n--- Testing Dataset Sample #{idx} ---")
        try:
            analyzer = DataAnalyzer(sample)
            summary = analyzer.get_summary()
        except TypeError as ex:
            print(f"[Type Error] {ex}")
        except ValueError as ex:
            print(f"[Value Error] {ex}")
        except EmptyDatasetError as ex:
            print(f"[Domain Error] {ex}")
        else:
            print("[Success] Summary Metrics:")
            for metric, val in summary.items():
                print(f"  - {metric.capitalize()}: {val}")
        finally:
            print("Processing complete.")
```

Question 3 (20 Marks): Data Visualization Mechanics & Bounds Guardrails

(a) [5 Marks] Plotting Architecture

Explain the role of figure initialization (`plt.figure()`), plot generation, axis labeling (`plt.xlabel()` , `plt.ylabel()`), and legend rendering.

(b) [5 Marks] Bounds Validation

Why must data dimensions match across coordinate axes (e.g., (X)-axis length equal to (Y)-axis length) before rendering? What exception is triggered if they differ?

© [5 Marks] Simple Baseline Solution: Basic Line Plotter

```
import matplotlib.pyplot as plt

x_vals =
y_vals =

plt.plot(x_vals, y_vals, label="Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales ($)")
plt.title("Monthly Sales Report")
plt.legend()
plt.show()
```

(d) [5 Marks] Complex Improvement: Object-Oriented Visualizer with Dimension Mismatch Handling

```
import matplotlib
matplotlib.use('Agg')  # Non-interactive backend suitable for server/sandbox environm
import matplotlib.pyplot as plt

class DimensionMismatchError(Exception):
    """Custom exception raised when X and Y coordinates differ in length."""
    def __init__(self, len_x, len_y):
        self.len_x = len_x
        self.len_y = len_y
        super().__init__(f"Coordinate length mismatch: X has {len_x} elements, but Y

class DatasetVisualizer:
    def __init__(self, x_data, y_data, title="Data Trend"):
        self.x_data = []
        self.y_data = []
        self.title = title
        self.validate_and_load(x_data, y_data)

    def validate_and_load(self, x_data, y_data):
        if not isinstance(x_data, (list, tuple)) or not isinstance(y_data, (list, tup
```

```
                raise TypeError("X and Y coordinate datasets must be lists or tuples.")

            if len(x_data) != len(y_data):
                raise DimensionMismatchError(len(x_data), len(y_data))

            if len(x_data) == 0:
                raise ValueError("Coordinate datasets cannot be empty.")

            self.x_data = [float(x) for x in x_data]
            self.y_data = [float(y) for y in y_data]

        def render_plot(self, output_filepath):
            fig, ax = plt.subplots(figsize=(8, 4.5))

            ax.plot(self.x_data, self.y_data, color="#2980b9", marker="o", linestyle="-",
            ax.set_title(self.title, fontsize=14, fontweight="bold", pad=12)
            ax.set_xlabel("X Axis (Independent Parameter)", fontsize=10)
            ax.set_ylabel("Y Axis (Dependent Output)", fontsize=10)
            ax.grid(True, linestyle="--", alpha=0.6)
            ax.legend(loc="upper left")

            plt.tight_layout()
            plt.savefig(output_filepath, dpi=300)
            plt.close(fig)
            return output_filepath

    if __name__ == "__main__":
        x_test =
        y_test =

        try:
            viz = DatasetVisualizer(x_test, y_test, title="Engine Metric Output")
            out_path = viz.render_plot("/workspace/scratch/sample_plot.png")
        except TypeError as ex:
            print(f"[Type Error] {ex}")
        except DimensionMismatchError as ex:
            print(f"[Dimension Error] {ex}")
        except ValueError as ex:
            print(f"[Value Error] {ex}")
        except Exception as ex:
            print(f"[Unexpected Error] {ex}")
        else:
            print(f"[Success] Plot successfully generated and saved to: {out_path}")
        finally:
            print("Visualization routine finished.")
```

Question 4 (20 Marks): Tkinter GUI Integration with Statistical Calculators

(a) [5 Marks] Tkinter Layout Selection

Compare the `.grid()` and `.pack()` geometry managers when building a form that accepts input parameters and displays computed data metrics.

(b) [5 Marks] GUI Exception Guarding

Explain why input string extraction from `Entry` widgets must be wrapped inside `try/except` blocks before performing arithmetic operations.

© [5 Marks] Simple Baseline Solution: Tkinter Calculator

```
import tkinter as tk

def compute():
    val1 = float(entry1.get())
    val2 = float(entry2.get())
    result_label.config(text=f"Sum: {val1 + val2}")

root = tk.Tk()
root.title("Simple Adder")

entry1 = tk.Entry(root)
entry1.pack(pady=5)

entry2 = tk.Entry(root)
entry2.pack(pady=5)

btn = tk.Button(root, text="Add", command=compute)
btn.pack(pady=5)

result_label = tk.Label(root, text="Sum: --")
result_label.pack(pady=5)

root.mainloop()
```

(d) [5 Marks] Complex Improvement: Modular OOP Tkinter Application with `Entry`, `Listbox`, `messagebox`, and Exception Handling

```
import tkinter as tk
from tkinter import messagebox

class DataEntryStudioApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Statistical Data Entry Studio")
        self.geometry("450x380")
        self.config(bg="#f4f6f7")

        self.dataset = []
        self.build_ui()

    def build_ui(self):
        # Header Panel
        header_frame = tk.Frame(self, bg="#2c3e50", pady=10)
        header_frame.pack(side=tk.TOP, fill=tk.X)

        tk.Label(
            header_frame,
            text="Data Entry & Processing Studio",
            font=("Arial", 14, "bold"),
            fg="white",
            bg="#2c3e50"
        ).pack()

        # Entry Form Panel (Grid Layout)
        form_frame = tk.LabelFrame(self, text=" Add Numerical Entry ", font=("Arial",
        form_frame.pack(pady=10, fill=tk.X, padx=15)

        tk.Label(form_frame, text="Value:", bg="#f4f6f7").grid(row=0, column=0, stick
        self.entry_val = tk.Entry(form_frame, width=15, font=("Arial", 10))
        self.entry_val.grid(row=0, column=1, padx=5)
        self.entry_val.focus()

        btn_add = tk.Button(
            form_frame,
            text="Add Point",
            command=self.add_value,
            bg="#27ae60",
            fg="white",
            font=("Arial", 9, "bold")
        )
        btn_add.grid(row=0, column=2, padx=10)

        # Dataset Display Listbox
```

```
        list_frame = tk.Frame(self, bg="#f4f6f7")
        list_frame.pack(pady=5, fill=tk.BOTH, expand=True, padx=15)

        self.scrollbar = tk.Scrollbar(list_frame, orient=tk.VERTICAL)
        self.listbox = tk.Listbox(
            list_frame,
            font=("Courier", 10),
            selectmode=tk.SINGLE,
            yscrollcommand=self.scrollbar.set
        )
        self.scrollbar.config(command=self.listbox.yview)

        self.listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Footer Action Panel
        footer_frame = tk.Frame(self, bg="#f4f6f7")
        footer_frame.pack(side=tk.BOTTOM, fill=tk.X, pady=10, padx=15)

        tk.Button(
            footer_frame,
            text="Compute Summary",
            command=self.compute_summary,
            bg="#2980b9",
            fg="white",
            font=("Arial", 9, "bold")
        ).pack(side=tk.LEFT, padx=5)

        tk.Button(
            footer_frame,
            text="Clear Dataset",
            command=self.clear_dataset,
            bg="#c0392b",
            fg="white",
            font=("Arial", 9, "bold")
        ).pack(side=tk.RIGHT, padx=5)

    def add_value(self):
        """Extracts input, validates float conversion, and updates Listbox."""
        raw_str = self.entry_val.get().strip()
        try:
            if not raw_str:
                raise ValueError("Input value cannot be empty.")

            val = float(raw_str)
            self.dataset.append(val)
```

```
            self.listbox.insert(tk.END, f"Data Point #{len(self.dataset)}: {val:.2f}"

            self.entry_val.delete(0, tk.END)
            self.entry_val.focus()

        except ValueError as ex:
            messagebox.showerror("Input Format Error", f"Invalid input: {ex}")

    def compute_summary(self):
        """Computes summary statistics on current dataset."""
        try:
            if not self.dataset:
                raise ValueError("Dataset is empty. Add data points first.")

            count = len(self.dataset)
            total = sum(self.dataset)
            mean = total / count
            minimum = min(self.dataset)
            maximum = max(self.dataset)

            summary_msg = (
                f"Count: {count}\n"
                f"Sum: {total:.2f}\n"
                f"Mean: {mean:.2f}\n"
                f"Min: {minimum:.2f}\n"
                f"Max: {maximum:.2f}"
            )
            messagebox.showinfo("Dataset Summary", summary_msg)

        except ValueError as ex:
            messagebox.showwarning("Processing Warning", str(ex))

    def clear_dataset(self):
        """Clears in-memory dataset and UI Listbox."""
        if self.dataset:
            if messagebox.askyesno("Confirm Clear", "Are you sure you want to clear a
                self.dataset.clear()
                self.listbox.delete(0, tk.END)

if __name__ == "__main__":
    app = DataEntryStudioApp()
    app.mainloop()
```

Question 5 (20 Marks): Capstone — Modular OOP Data & Plotting Studio with Custom Exceptions

(a) [5 Marks] Architectural Design

Explain how object-oriented encapsulation protects data integrity when building an end-to-end data acquisition and plotting pipeline.

(b) [5 Marks] Domain Exception Declarations

Write the code definition for a custom exception named `OutOfRangeDataError` that tracks out-of-bounds parameters.

© [10 Marks] Capstone Implementation

Write a complete, continuous Python application combining OOP, custom exception handling, statistical processing, and a Tkinter GUI.

```
import tkinter as tk
from tkinter import messagebox

# --- Custom Exception Class ---
class OutOfRangeDataError(Exception):
    """Custom exception raised when input value breaches allowed physical domain bour
    def __init__(self, value, min_bound, max_bound):
        self.value = value
        self.min_bound = min_bound
        self.max_bound = max_bound
        self.message = f"Value {value} is out of permitted bounds [{min_bound}, {max_
        super().__init__(self.message)

# --- Modular Data Model Class ---
class SensorDataModel:
    MIN_PERMITTED = 0.0
    MAX_PERMITTED = 100.0

    def __init__(self):
        self._records = []

    def add_reading(self, raw_input):
        """Validates and appends a sensor reading."""
        if not raw_input or not raw_input.strip():
            raise ValueError("Input field cannot be empty.")

        val = float(raw_input.strip())
```

```
            if val < SensorDataModel.MIN_PERMITTED or val > SensorDataModel.MAX_PERMITTEI
                raise OutOfRangeDataError(val, SensorDataModel.MIN_PERMITTED, SensorDataM

            self._records.append(val)
            return val

        def get_records(self):
            return list(self._records)

        def compute_stats(self):
            if not self._records:
                raise ValueError("No data records available to process.")

            n = len(self._records)
            avg = sum(self._records) / n
            return n, avg, min(self._records), max(self._records)

        def clear(self):
            self._records.clear()

    # --- Full GUI Application Class ---
    class CapstoneDataStudioApp(tk.Tk):
        def __init__(self):
            super().__init__()
            self.title("Capstone Sensor Analytics Studio")
            self.geometry("500x420")
            self.config(bg="#ebedef")

            self.model = SensorDataModel()
            self.build_gui()

        def build_gui(self):
            # Title Banner
            title_frame = tk.Frame(self, bg="#1b2631", pady=10)
            title_frame.pack(side=tk.TOP, fill=tk.X)

            tk.Label(
                title_frame,
                text="Sensor Analytics & Validation Studio",
                font=("Helvetica", 14, "bold"),
                fg="#f1c40f",
                bg="#1b2631"
            ).pack()

            # Form Section
```

```
form_box = tk.LabelFrame(self, text=" Input Sensor Reading ", font=("Arial",
form_box.pack(pady=10, fill=tk.X, padx=15)

tk.Label(form_box, text="Value (0.0 - 100.0):", bg="#ebedef").grid(row=0, co
self.ent_value = tk.Entry(form_box, width=15, font=("Arial", 10))
self.ent_value.grid(row=0, column=1, padx=5)
self.ent_value.focus()

btn_submit = tk.Button(
    form_box,
    text="Record Reading",
    command=self.handle_add_reading,
    bg="#27ae60",
    fg="white",
    font=("Arial", 9, "bold")
)
btn_submit.grid(row=0, column=2, padx=10)

# Listbox Frame
list_box_frame = tk.Frame(self, bg="#ebedef")
list_box_frame.pack(pady=5, fill=tk.BOTH, expand=True, padx=15)

self.scrollbar = tk.Scrollbar(list_box_frame, orient=tk.VERTICAL)
self.listbox = tk.Listbox(
    list_box_frame,
    font=("Courier", 10),
    yscrollcommand=self.scrollbar.set
)
self.scrollbar.config(command=self.listbox.yview)

self.listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

# Control Footer
footer = tk.Frame(self, bg="#ebedef")
footer.pack(side=tk.BOTTOM, fill=tk.X, pady=10, padx=15)

tk.Button(
    footer,
    text="Process Statistics",
    command=self.handle_compute,
    bg="#2980b9",
    fg="white",
    font=("Arial", 9, "bold")
).pack(side=tk.LEFT, padx=5)
```

```
        tk.Button(
            footer,
            text="Reset System",
            command=self.handle_reset,
            bg="#c0392b",
            fg="white",
            font=("Arial", 9, "bold")
        ).pack(side=tk.RIGHT, padx=5)

    def handle_add_reading(self):
        """Handles input validation, catches custom OutOfRangeDataError exceptions."'
        raw_val = self.ent_value.get()
        try:
            val = self.model.add_reading(raw_val)
            count = len(self.model.get_records())
            self.listbox.insert(tk.END, f"Reading #{count:02d}: {val:>6.2f} units")

            self.ent_value.delete(0, tk.END)
            self.ent_value.focus()

        except ValueError as ex:
            messagebox.showerror("Input Error", f"Format Error: {ex}")
        except OutOfRangeDataError as ex:
            messagebox.showwarning("Domain Bounds Violation", str(ex))
        except Exception as ex:
            messagebox.showerror("System Error", f"Unhandled exception: {ex}")

    def handle_compute(self):
        try:
            n, avg, min_v, max_v = self.model.compute_stats()
            summary = (
                f"Total Samples: {n}\n"
                f"Average Value: {avg:.2f}\n"
                f"Minimum Value: {min_v:.2f}\n"
                f"Maximum Value: {max_v:.2f}"
            )
            messagebox.showinfo("Analytics Report", summary)
        except ValueError as ex:
            messagebox.showwarning("Analytics Alert", str(ex))

    def handle_reset(self):
        if self.model.get_records():
            if messagebox.askyesno("Confirm System Reset", "Purge all recorded data p
                self.model.clear()
                self.listbox.delete(0, tk.END)
```

```
if __name__ == "__main__":
    app = CapstoneDataStudioApp()
    app.mainloop()
```